

ShortStop — Report 2

Tìm hiểu kĩ về khung P-R-C-S và Kết quả Thực nghiệm trên Reach-Avoid-2D

Tamira Le

Ngày 31 tháng 8 năm 2026

Track A & Track B — Kế hoạch song song (cập nhật)

Track A & Track B: lộ trình song song — cập nhật (1/3)

Track A	Status	Track B	Status
Stage 0 — Reachability cơ bản (interval/zonotope, Minkowski sum, Lipschitz) + Shielding/Safe RL context; Report 2 Mục 2, [44],[45].	Đã xong	Setup — env Reach-Avoid-2D, interval arithmetic, cvxpy; policy Gaussian giả lập chỉ để dựng khung.	Đã xong
Stage 1 — Reachtube & Assumption 1 (sound over-approximation); Report 2 Mục 2, Eq. 1.	Đã xong	Reach-only shield — reachtube, binary reject (Gaussian; violation 78.4%→0.2%, success 21.6%→23.0%; Table 1).	Đã xong
Stage 2 — STL syntax & robustness-to-go, Assumption 2; Report 2 Mục 3, Eq. 2; còn thiếu $\diamond/\mathcal{U}$.	Đã xong	STL-to-go — thay reject nhị phân bằng robustness lower bound (Gaussian; violation 0.4%, prec. 12.0%; Table 1).	Đã xong
Stage 3 — CEGIS: tìm counterexample (adversarial search); Report 2 Mục 4, Eq. 3, [10]–[15].	Đã xong	CE search — box-vs-circle closed form trên 2D (Gaussian; violation 0.4%, cùng số Stage 2; Table 1 & 3).	Đã xong

Track A & Track B: lộ trình song song — cập nhật (2/3)

Track A	Status	Track B	Status
Stage 4 — CEGIS: sửa theo counterexample (trust-region projection); Report 2 Mục 4, Eq. 4–5. [†]	Đã xong	Repair — Algorithm 1 đầy đủ (Gaussian; violation 0.2%, success 38.4%, precision 71.6%; Table 1).	Đã xong
Stage 5 — MPC/CBF & fallback, Assumption 3 (safe fallback tồn tại). [‡]	Đang tìm hiểu	Baselines — MPC-Filter, CBF-Shield, STL-Monitor, Conf-Thresh (Gaussian; 4/4 đã chạy; Table 2).	Đã xong
Stage 6a — Generative policy, bản nhỏ (Diffusion Policy/flow matching).	Chưa bắt đầu	2D policy thật — train một diffusion/flow policy nhỏ trên Reach-Avoid-2D, hoặc import policy có sẵn.	Chưa bắt đầu
Stage 6b — Safety guarantee (bounded-time → all-time, Grönwall trade-off); Report 2 mới nhắc lại, chưa tự chứng minh.	Chưa bắt đầu	Metrics + stress test — 7 metric, horizon sweep, bootstrap, chạy lại với policy 2D thật thay Gaussian.	Chưa bắt đầu

[†] Vòng lặp `max_repair_iters > 1` không có convergence guarantee — an toàn chỉ giữ vững nhờ re-certify, chưa viết thành proposition riêng. [‡] Đang đọc Wabersich & Zeilinger [33], Ames et al. [37],[38], Cheng et al. [39] (READING_LIST.md mục A); chưa đào sâu bằng khuôn trích dẫn+công thức+code như Mục 1–4.

Track A & Track B: lộ trình song song — cập nhật (3/3)

Track A	Status	Track B	Status
Stage 7a — Generative policy, scale lên (cùng nhóm với 6a).	Chưa bắt đầu	LIBERO — swap sang env thật, long-horizon manipulation.	Chưa bắt đầu (tùy resource GPU)
Stage 7b — Generative policy, scale lên (cùng nhóm với 6a).	Chưa bắt đầu	ManiSkill2 — swap sang env thật, contact-rich precision.	Chưa bắt đầu (tùy resource GPU)
Stage 7c — Generative policy, scale lên (cùng nhóm với 6a).	Chưa bắt đầu	RLBench — swap sang env thật, diverse manipulation tasks.	Chưa bắt đầu (tùy resource GPU)

So với Report 1: Stage 1–4 (cả Track A và Track B) và code của 4 baseline (Track B, Stage 5) đã chuyển từ “chưa bắt đầu” sang **đã xong**. Lý thuyết gốc của baseline (Track A, Stage 5) là mảnh còn lại gần nhất, đang **tìm hiểu**; Stage 6a trở đi vẫn y nguyên so với Report 1.

Phần I — Baseline methods

Sáu hệ thống so sánh (paper Table II) — trạng thái tái lập

Hệ thống	Cơ chế	Nguồn	Trạng thái
Unshielded	policy nguyên bản, không lọc	—	Đã chạy (Reach-Avoid-2D)
Conf-Thresh	reject theo ensemble disagreement	paper Sec. VI-D	Đã chạy, nhưng proxy yếu [†]
MPC-Filter	QP sửa toàn horizon, tuyến tính hoá	Wabersich & Zeilinger [33]	Đã chạy
CBF-Shield	barrier QP $h(x) = \ x - c\ ^2 - r^2$	Ames et al. [38], Cheng et al. [39]	Đã chạy
STL-Monitor	STL trên nominal rollout, không tube	Deshmukh et al. [53]	Đã chạy
ShortStop (Stage 1–4)	Reach + STL + CE search + Repair	phương pháp của paper	Đã chạy đầy đủ

[†] ConfThreshShield dùng spread quanh centroid của K endpoint làm proxy “ensemble disagreement” vì GaussianChunkPolicy không phải ensemble thật — đo được là proxy này *không mang tín hiệu* về nguy hiểm trên scenario hiện tại (violation phẳng $\approx 81\%$ ở mọi threshold), khác paper (real diffusion ensemble: 11.2% violation, tín hiệu yếu nhưng có thật). Xem `docstring shortstop/baselines.py:ConfThreshShield`.

Trạng thái tổng thể: IN PROGRESS

Tất cả 6 hệ thống mới chạy trên Reach-Avoid-2D + policy Gaussian giả lập, cùng $\hat{f}/\varepsilon$ /fallback để so sánh công bằng (đúng nguyên tắc paper). Chưa chạy trên diffusion/flow policy thật, và chưa chạy trên LIBERO/ManiSkill2/RLBench — khớp Stage 6a/7a–7c (Track B, Report 1) vẫn “chưa bắt đầu.”

Phần II — Implementation: kết quả thực nghiệm

Thiết lập chung cho 3 bảng dưới đây

- **Env:** Reach-Avoid-2D, start/goal cố định, 3 obstacle tròn vị trí/bán kính ngẫu nhiên mỗi episode (`shortstop/experiment.py:make_scenario`).
- **Policy:** GaussianChunkPolicy, $K = 8$, $H = 8$ (stand-in cho π_θ thật).
- **Calibration:** $\bar{w}$ certify = quantile 99% $\times 1.25$ trên 200 episode held-out (`shortstop/calibration.py`), không phải ground-truth $\bar{w} = 0.02$ cho không.
- **Table 1 & 2:** 500 episodes/hàng, cùng seed offset — so sánh trực tiếp được với nhau.
- **Table 3 (optional):** 300 episodes/hàng, chạy riêng để quét `max_repair_iters` — không so số tuyệt đối với Table 1/2, chỉ so nội bộ 3 hàng với nhau.

Table 1 — Thêm từng stage vào (results/ablation.json)

Stage	Viol.%↓	Succ.%↑	Act.%	Lat.(ms)	Prec.↑	Recov.↑	Cons.%↓
Unshielded	78.4	21.6	0.0	n/a	n/a	n/a	n/a
+ Reach-only (St.1)	0.2	23.0	75.4	0.77	22.2	n/a	17.6
+ STL-to-go (St.2)	0.4	20.4	78.2	1.04	12.0	n/a	24.1
+ CE search (St.3)	0.4	20.4	78.2	1.69	12.0	n/a	24.1
+ Repair (St.4, SHORTSTOP)	0.2	38.4	60.3	3.42	71.6	14.6	13.9

Table 2 — So với baseline (results/baselines.json)

Hệ thống	Viol.%↓	Succ.%↑	Act.%	Lat.(ms)	Prec.↑	Recov.↑	Cons.%↓
Unshielded	78.4	21.6	0.0	n/a	n/a	n/a	n/a
Conf-Thresh	79.0	21.0	46.5	0.19	13.5	n/a	18.5
MPC-Filter [33]	2.2	90.2	94.6	17.94	31.2	91.0	0.0
CBF-Shield [38,39]	18.0	82.0	48.1	4.92	47.4	100.0	0.0
STL-Monitor [53]	12.4	41.6	55.9	0.87	100.0	n/a	5.6
SHORTSTOP (Stage 4)	0.2	38.4	60.3	3.42	71.6	14.6	13.9

Table 3 (optional) — Đang tuning: quét max_repair_iters (stage34_iters_comparison.json)

Cấu hình (n=300)	Viol.%	Succ.%↑	Act.%	Lat.(ms)	Prec.	Recov.	Cons.%
CE search (St.3, không repair)	0.3	24.0	73.1	1.64	46.4	n/a	15.6
Repair, iters=1 (mặc định)	1.0	48.7	52.4	3.35	91.0	15.9	4.7
Repair, iters=3	1.7	52.3	47.3	6.71	93.9	19.1	4.7

- Ý nghĩa của bảng này: max_repair_iters=1 là Algorithm 1 gốc (một bước gradient, không retry); iters=3 là mở rộng CEGIS-style (lập tìm counterexample mới sau mỗi lần sửa thất bại) — vượt ngoài paper, đang được thử nghiệm.
- Tăng iters 1→3: success 48.7% → 52.3%, precision 91.0% → 93.9%, recovery 15.9% → 19.1% — đổi lại latency tăng gần gấp đôi (3.35 → 6.71ms) và violation nhích nhẹ (1.0% → 1.7%, vẫn rất thấp so mọi baseline ở Table 2).
- Đây chính là hướng thu hẹp khoảng cách với MPC-Filter (Table 2: success 90.2%) mà không cần đổi thuật toán — gợi ý còn dư địa tuning (step_size, trust_region, max_repair_iters) trước khi cần một cơ chế repair mạnh hơn.
- Lưu ý: bảng này chạy 300 episodes/seed riêng (không phải 500 như Table 1–2) — chỉ dùng để so sánh nội bộ 3 hàng, không đối chiếu tuyệt đối với Table 1–2.

Phần III Chuẩn bị hạ tầng: Linux cho LIBERO/ManiSkill2/RLBench

Đã có policy pretrained cho 3 benchmark chưa?

Benchmark	Checkpoint public?	Nguồn / ghi chú
LIBERO	Có	π_0 (Physical Intelligence, flow-matching) fine-tune trên LIBERO-Spatial/Object/Goal/10; host qua repo <code>openpi</code>
ManiSkill2/3	Chỉ có code + demo data	Official repo có script train BC/Diffusion Policy/ACT + 4M+ frame demo, chưa thấy checkpoint sẵn để tải; có hỗ trợ eval checkpoint ngoài (Octo, RT-X, RDT-1B)
RLBench	Có, nhưng sai kiểu policy	PerAct (18 task) và RVT/RVT-2 (20 task) có checkpoint chính thức — nhưng cả hai là transformer dự đoán 1 keyframe action, <i>không phải</i> generative chunk policy mà π_θ của SHORTSTOP giả định

- **LIBERO** là ứng viên tốt nhất cho Stage 7a: π_0 đúng kiểu flow-matching sinh action chunk — khớp thẳng giả định π_θ của SHORTSTOP, không cần train lại từ đầu.
- **ManiSkill2/3**: hướng khả thi nhất là tự train Diffusion Policy bằng script + demo data chính thức của ManiSkill, hoặc thử adapt Octo/RDT-1B nếu action space khớp.
- **RLBench**: PerAct/RVT tốt để so sánh baseline “học từ demo”, nhưng để đúng khung generative-policy của SHORTSTOP, cần policy khác (VD “Mini Diffuser,” arXiv:2505.09430 — diffusion policy multitask cho RLBench-18) thay vì PerAct/RVT.

Cần double-check khi setup thực tế

Link/checkpoint/license có thể đã đổi theo thời gian — luôn vào đúng repo (`openpi`, ManiSkill, `peract/peract`, NVlabs/RVT) xác nhận lại trước khi setup: format checkpoint, version dependency, và điều khoản sử dụng (đặc biệt π_0 host qua S3 của Physical Intelligence).

Vì sao Stage 7a–7c không chạy được như prototype 2D

- Toàn bộ Stage 0–4 + baseline (Report 1 & 2) chạy tốt trên Windows + venv Python thuần (numpy/cvxpy) — không cần OS đặc biệt, vì Reach-Avoid-2D không có physics/rendering engine compiled nào cả.
- LIBERO/ManiSkill2/RLBench (Stage 7a–7c) khác hẳn: mỗi benchmark bọc quanh một physics/rendering engine biên dịch sẵn, và official install của cả 3 đều nhắm Ubuntu:
 - **LIBERO**: robosuite + MuJoCo — offscreen renderer (EGL/OSMesa) và một số dependency compiled chỉ có wheel Linux ổn định.
 - **ManiSkill2**: SAPIEN — thể mạnh chính là GPU-parallel simulation (hàng trăm/ngày env song song trên 1 GPU) qua Vulkan + driver NVIDIA Linux; không có tương đương ổn định trên Windows.
 - **RLBench**: CoppeliaSim qua PyRep — CoppeliaSim có build Windows, nhưng toàn bộ tooling/community của RLBench giả định Ubuntu (đặc biệt cho training headless trên server không màn hình).
- Train diffusion/flow policy (Stage 6a–7c) cũng cần CUDA thật cho GPU — không tách rời được khỏi nhu cầu "có một môi trường Linux ổn định".

So sánh 4 hướng setup Linux

Hướng	GPU/CUDA training	Công sức setup	Reproducibility
1. Dual Boot	Tốt nhất (native)	Cao	Trung bình
2. VM chia RAM (VirtualBox/VMware)	Kém/không có	Thấp	Trung bình
3. Docker (Desktop, WSL2 backend)	Tốt	Trung bình	Cao nhất
4. WSL2 trực tiếp (không qua Docker)	Tốt	Thấp nhất	Trung bình

- **1. Dual Boot** — GPU chạy native, hiệu năng tốt nhất; trên máy cá nhân đây là lựa chọn hợp lý cho phần train nặng (Stage 7a–7c) — vẫn nên backup dữ liệu trước khi chia lại ổ đĩa, và chấp nhận phải reboot để đổi qua lại Windows/Linux.
- **2. VM chia RAM kiểu VirtualBox/VMware Workstation** — hypervisor desktop thường *không* hỗ trợ GPU passthrough cho CUDA training thật, chỉ chạy CPU trong VM — không giải quyết được nhu cầu chính (train policy cần GPU).
- **3. Docker (Desktop, WSL2 backend) + NVIDIA Container Toolkit** — hỗ trợ CUDA khá mature từ ~2021, và nhiều benchmark có sẵn Dockerfile chính thức (pin đúng version) $\Rightarrow$ reproducibility cao nhất. Docker Desktop free cho cá nhân/người nghiên cứu; nếu dùng cho công việc gắn với công ty lớn, licensing của Docker dựa trên *cách dùng* hơn là máy nào — nên tự kiểm tra điều khoản hiện hành nếu không chắc.
- **4. WSL2 trực tiếp** — về bản chất vẫn là 1 VM nhẹ (Hyper-V), nhưng Microsoft+NVIDIA xây riêng CUDA passthrough chính thức cho nó (khác hẳn VM desktop ở mục 2) — setup chỉ cần `wsl -install`, không đụng tới ổ đĩa/dual-boot. Rendering offscreen (EGL) cho robosuite/CoppeliaSim đôi khi cần thêm cấu hình WSLg.

Đề xuất cụ thể theo giai đoạn

- **Giai đoạn hiện tại** (Stage 6a, policy nhỏ trên Reach-Avoid-2D): giữ nguyên Windows + venv — chưa cần Linux gì cả.
- **Khi bắt đầu Stage 7a–7c** (LIBERO/ManiSkill2/RLBench thật), trên máy cá nhân:
 1. **WSL2 trước tiên** để thử nhanh — cài Ubuntu 22.04 (có CUDA, không cần reboot), test riêng từng benchmark: LIBERO/RLBench nhiều khả năng cài được ngay; ManiSkill2 (GPU-parallel qua Vulkan) cần kiểm tra kỹ trong WSL2 trước khi tin tưởng.
 2. Nếu WSL2 đủ ổn (đặc biệt cho LIBERO π_0 checkpoint, xem slide trước) — **có thể dừng ở đây**, không cần dual-boot.
 3. Nếu vướng driver/rendering (nhiều khả năng nhất ở ManiSkill2's Vulkan), hoặc muốn hiệu năng full cho train dài hạn: **dual boot** sang Ubuntu — máy cá nhân nên đây hoàn toàn khả thi, không còn ràng buộc IT như suy nghĩ ban đầu.
 4. Nếu máy cá nhân không đủ GPU mạnh cho training thật: thuê **cloud GPU instance Ubuntu** (AWS/GCP/Lambda Labs...) — trả phí theo giờ, tránh phải đầu tư phần cứng.
 5. **Docker** dùng để đóng gói môi trường reproducible *sau khi* đã có Linux (WSL2, dual-boot, hoặc cloud instance) — không phải giải pháp thay cho "có Linux", mà là lớp đóng gói thêm vào.
- **Bỏ qua VM** chia RAM kiểu VirtualBox/VMware cho mục đích train GPU — chỉ hợp để đọc code/test phần không cần GPU.

Phần IV — Đào sâu khung P-R-C-S

Nhắc lại: một decision step trong P-R-C-S

Ghi chú tên gọi

Khung 4 bước của SHORTSTOP được paper gọi là **P-R-C-S** (Propose $\rightarrow$ Reach $\rightarrow$ Certify $\rightarrow$ Select/Repair) — dùng xuyên suốt Report 1 và toàn bộ code. Report này đào sâu từng chữ cái thành một mục riêng bên dưới.

$$\pi_{\theta}(o_t) \xrightarrow{\mathbf{P}} \{a^{(i)}\}_{i=1}^K \xrightarrow{\mathbf{R}} \hat{\mathcal{R}}_{0:H}^{(i)} \xrightarrow{\mathbf{C}} \rho(\varphi, \hat{\mathcal{R}}^{(i)}) \xrightarrow{\mathbf{S}} a^* \text{ hoặc } \pi_{fb}$$

- **Không sửa** π_{θ} : shield chỉ quyết định chunk nào (gốc hoặc đã repair) được thực thi.
- Mỗi mục dưới đây theo đúng khuôn: **Trích dẫn** $\rightarrow$ **Công thức (paper)** $\rightarrow$ **Mapping code**.

Mục 1 — (P) Propose: đề xuất K candidate chunk

Trích dẫn. π_θ là generative policy học từ hai cơ chế phổ biến: C. Chi et al., “*Diffusion Policy: Visuomotor Policy Learning via Action Diffusion*,” RSS 2023 [1]; Y. Lipman et al., “*Flow Matching for Generative Modeling*,” ICLR 2023 [4].

Công thức (paper, Sec. III-C, trước Eq. 1).

$$\{a^{(i)}\}_{i=1}^K \sim \pi_\theta(\cdot \mid o_t), \quad g(a^{(i)}) = \text{task-progress score}.$$

K nhỏ (paper dùng $K = 8$) — “shield lọc, không tìm kiếm toàn bộ action space.”

Mapping code.

- `shortstop/policy.py:GaussianChunkPolicy.propose()` — *stand-in* Gaussian cho π_θ thật (chưa có diffusion/flow policy, xem Stage 6a của Report 1), sinh $K = \text{n_candidates}$ chunk quanh một reference velocity.
- Gọi từ `shortstop/experiment.py:run_episode()`, dòng `candidates = policy.propose(state)`.
- $g(a)$ hiện là `--goal_distance` (mỗi `_score()` trong `shield.py/baselines.py`) — đơn giản hoá: chưa dùng policy likelihood hay learned value như paper cho phép.

Mục 2 — (R) Reach: lan truyền reachtube sound

Trích dẫn. Soundness contract của set-propagation — nguồn của Assumption 1: M. Althoff, “An Introduction to CORA 2015,” ARCH 2015 [44]; X. Chen, E. Ábrahám, S. Sankaranarayanan, “Flow*: An Analyzer for Non-Linear Hybrid Systems,” CAV 2013 [45].

Công thức Eq. (1).

$$\hat{\mathcal{R}}_{k+1} = \hat{f}(\hat{\mathcal{R}}_k, a_{t+k}) \oplus \mathcal{B}(\varepsilon_k + \bar{w}), \quad \varepsilon_k = \sup_{x \in \hat{\mathcal{R}}_k} \varepsilon(x, a_{t+k}).$$

Mapping code (shortstop/reach.py).

- class Box — interval/axis-aligned box, thay cho zonotope đầy đủ của paper (low, high, inflate() = $\oplus \mathcal{B}(r)$, shift() = ảnh của $\hat{f}$).
- propagate(box, action, dt, w_bar, model_error) — đúng 1 bước Eq. (1); $\hat{f}$ ở đây là point-mass model $x_{t+1} = x_t + a dt$ nên model_error=0 trong Phase 1 (vì $\hat{f} \equiv f$ chính xác, không có sai số mô hình thật).
- propagate_tube(x0, actions, dt, w_bar) — lặp propagate để dựng cả $\hat{\mathcal{R}}_{0:H}$ từ một điểm khởi đầu.
- $\bar{w}$ không lấy ground-truth mà được shortstop/calibration.py:calibrate_w_bar() ước lượng từ quantile 99% residual thực đo được (đúng recipe Table VII của paper).

Mục 3 — (C) Certify: STL robustness-to-go trên tube

Trích dẫn. O. Maler, D. Nickovic, “*Monitoring Temporal Properties of Continuous Signals*,” FORMATS 2004 [47]; A. Donzé, O. Maler, “*Robust Satisfaction of Temporal Logic over Real-Valued Signals*,” FORMATS 2010 [48]; G. Fainekos, G. Pappas [49] (soundness của STL quantitative semantics, dùng trong Theorem 1); J. Deshmukh et al., “*Robust Online Monitoring of STL*,” FMSD 2017 [53] — interval semantics cụ thể (Assumption 2), cũng chính là nguồn của baseline STL-Monitor.

Công thức Eq. (2).

$$\rho(\varphi, \hat{\mathcal{R}}) = \min_{0 \leq k \leq H} \inf_{x \in \hat{\mathcal{R}}_k} \text{dist}(x, X_u).$$

Mapping code (shortstop/stl.py).

- `step_robustness(box, obstacles)` — $\inf_{x \in \text{box}} \text{dist}(x, X_u)$; closed-form vì X_u là hợp các hình tròn $\Rightarrow$ quy về `box.closest_point(center)`.
- `robustness_to_go(tube, obstacles)` — min qua $k = 1..H$ (bỏ $\hat{\mathcal{R}}_0$ vì đã "realized", cùng convention với `ReachOnlyShield`).
- Test chấp nhận: `shortstop/shield.py:STLShield._admissible()` $\Leftrightarrow \rho \geq \varepsilon$ (margin, EPSILON=0.05 trong `run_ablation.py`) — khác Stage 1 (`ReachOnlyShield`) vốn chỉ nhị phân `intersects_any()`.

Mục 4 — (S) Select/Repair, phần 1: tìm counterexample

Trích dẫn. A. Solar-Lezama et al., “Combinatorial Sketching for Finite Programs,” ASPLOS 2006 [10] (CEGIS gốc); E. Clarke et al., “Counterexample-Guided Abstraction Refinement,” CAV 2000 [11] (CEGAR); công cụ falsification CPS: S-TaLiRo [12], Breach [13], VerifAI [14], survey black-box safety validation [15].

Công thức Eq. (3).

$$(k^*, x^*) = \arg \min_{0 \leq k \leq H, x \in \hat{\mathcal{R}}_k} \text{dist}(x, X_u).$$

Paper giải bằng M bước projected-gradient/vertex sampling trên zonotope generators.

Mapping code.

- `shortstop/stl.py:find_counterexample(tube, obstacles)` — closed-form box-vs-circle search (thay numeric M -step search, vì bài toán ở đây có nghiệm đóng: box + disk); trả `step`, `obstacle`, `witness`, `robustness`.
- `shortstop/shield.py:CEShield._diagnose()` — gọi `find_counterexample` cho mọi candidate bị reject, lưu song song với `admissible_mask`. Đây là bước chuẩn bị cho repair (Stage 3 chưa tự sửa gì, chỉ định vị lỗi).

Mục 4 (tiếp) — (S) Select/Repair, phần 2: sửa + chọn

Công thức Eq. (4) — repair.

$$d = \nabla_a \text{dist}(\phi(x_t, a)_{k^*}, X_u) \Big|_{a=a^{(i)}}, \quad a' = \Pi_{A, \delta}(a^{(i)} + \eta d).$$

Công thức Eq. (5) — selection.

$$a^* = \arg \max_{i: \rho(\varphi, \mathcal{R}^{(i)}) \geq \varepsilon} g(a^{(i)}).$$

Mapping code (shortstop/shield.py:RepairShield).

- `_repair_direction()` — hướng d = witness – obstacle center, chuẩn hoá (fallback: hướng từ nominal-rollout point nếu witness trùng tâm obstacle).
- `_repair()` — 1 bước gradient η = `step_size`, chiếu $\Pi_{A, \delta}$ qua `_project_to_trust_region()` (δ = `trust_region`), rồi re-certify; `max_repair_iters`=1 khớp đúng Algorithm 1 dòng 7–13 (một bước, không retry) — > 1 là mở rộng CEGIS-style (lặp tìm counterexample mới), vượt ra ngoài paper.
- `select()` — vòng lặp Eq. (5) trên tập admissible (gốc + đã repair); nếu rỗng → fallback `np.zeros_like` (π_{fb} = zero action, đứng yên).
- Tham số paper (Table VII): $\eta = 0.05$, $\delta = 0.1$; prototype 2D dùng $\eta = 0.3$, $\delta = 1.0$ (tune riêng theo scale bài toán — xem `run_ablation.py` và docstring `RepairShield`). `max_repair_iters` chính là biến số của Table 3 (Phần III).

P-R-C-S $\rightarrow$ Assumption 1–3 $\rightarrow$ Theorem 1: chuỗi mapping đầy đủ

Bước	Assumption bảo vệ	Nếu sai thì hỏng ở đâu (code)
R (Reach)	Asm. 1: reachtube sound	reach.propagate: $\varepsilon_k + \bar{w}$ tính thiếu $\Rightarrow$ tube hẹp hơn thật
C (Certify)	Asm. 2: ρ là cận dưới đúng	stl.robustness_to_go: sai công thức Eq. (2) $\Rightarrow$ chứng nhận s
S (Select) khi $C = \emptyset$	Asm. 3: tồn tại π_{fb} hợp lệ	shield.select(): fallback hiện là zero action — chưa chứng m

Điểm cần Track A giải quyết tiếp (chưa xong, Report 1 Part 6)

Fallback `np.zeros_like` trong code hiện tại là *giả lược*: nó dừng robot lại, nhưng Assumption 3 đòi hỏi một π_{fb} với X_s robust-control-invariant thật sự — việc chứng minh "đứng yên = an toàn" trong Reach-Avoid-2D chưa được viết ra tường minh.